carla-track-control

NMPC and pure pursuit on a race track in CARLA, with CARLA or Project Chrono vehicle dynamics, and a standalone C++ controller library

Repository: https://github.com/Jharsh01/carla-track-control (branch main, commit 78eb70d) - report of 2026-10-15

Summary

The race-track scenario from the NMPC_CARLA project (a 1.29 km circuit with slower traffic) now runs in CARLA 0.9.15 with both controllers, the NMPC and the pure-pursuit baseline, and with a choice of who simulates the ego car: CARLA's own PhysX vehicle or the Project Chrono multibody sedan. The work was done as a new repository designed for two uses:

- **Clone and run on a cloud GPU.** On an NVIDIA Brev VM, `./setup.sh` installs everything (system packages, Python 3.8 environment, Project Chrono 9.0.1, CARLA 0.9.15, the build and the tests) and `./run.sh` starts CARLA and drives the track, writing a chase-camera video with a HUD for each controller, or streaming it live to a browser.

- **A publishable controller library.** The controllers live in `track_control/`, a standalone C++17 CMake project with install rules and a CMake package, its own tests and example, and no dependency on CARLA, Chrono or Python, ready to be split into its own repository.

Along the way the NMPC, which had spun the Chrono car in the first chicane, was made to lap it cleanly: measurements on the Chrono car showed two model errors (tire balance between the axles, and the effect of drive force on front grip), and both were fixed (section 6).

What is verified and what is not. Everything except a real CARLA server was run and checked: the library tests, offline laps on both plants (bicycle-plant laps identical sample for sample to the original project), the whole CARLA bridge against a stand-in carla module that follows CARLA's conventions, and `setup.sh` from scratch in a clean Ubuntu 22.04 container. CARLA itself was not run (by request, not on the laptop), so the first run on Brev is the first contact with a real server; section 9 lists what may need adjusting.

1. Can I pull the repo into a VM and just run the script?

Yes, that is what the repository is built for, with one caveat: the CARLA path has not met a real CARLA server yet, so treat the first run as a bring-up. The steps:

- Create a Brev GPU instance in **VM mode** (not container mode): CARLA refuses to run as root, and VM mode gives you the ubuntu user with sudo. Ubuntu 22.04, a GPU with 16 GB or more is a comfortable choice (L4, A10G; CARLA itself asks for 6-8 GB), about 60 GB of disk.

- Clone into the persistent workspace and run the two scripts:

```
cd ~/workspace
git clone https://github.com/Jharsh01/carla-track-control.git
cd carla-track-control
./setup.sh                    # about an hour: CARLA download (8.4 GB) + Chrono build
./run.sh                      # NMPC, then pure pursuit, on CARLA's vehicle dynamics
./run.sh --dynamics chrono    # the same with the Project Chrono sedan as the car
```

- Results land in `results/carla/`: `nmpc_carla.mp4`, `pure_pursuit_carla.mp4` (and `_chrono` variants), each with a .csv log and a .json summary. Copy them to the laptop with `scp '<instance>:~/workspace/carla-track-control/results/carla/*.mp4' .`

- To watch live instead: on the laptop `ssh -L 8080:localhost:8080 <instance>`, on the VM `./run.sh --controller nmpc --stream 8080`, then open `http://localhost:8080`.

- If the CARLA server fails to start, `scripts/start_carla.sh` prints the end of its log; a Vulkan/driver problem is the usual cause on headless VMs (check `vulkaninfo --summary`, `nvidia-smi`). `./run.sh --docker` starts CARLA from its official Docker image instead.

Without a CARLA server at all, `./setup.sh --no-carla` and `./run.sh --offline` run the same controllers on the offline C++ simulation.

2. The scenario

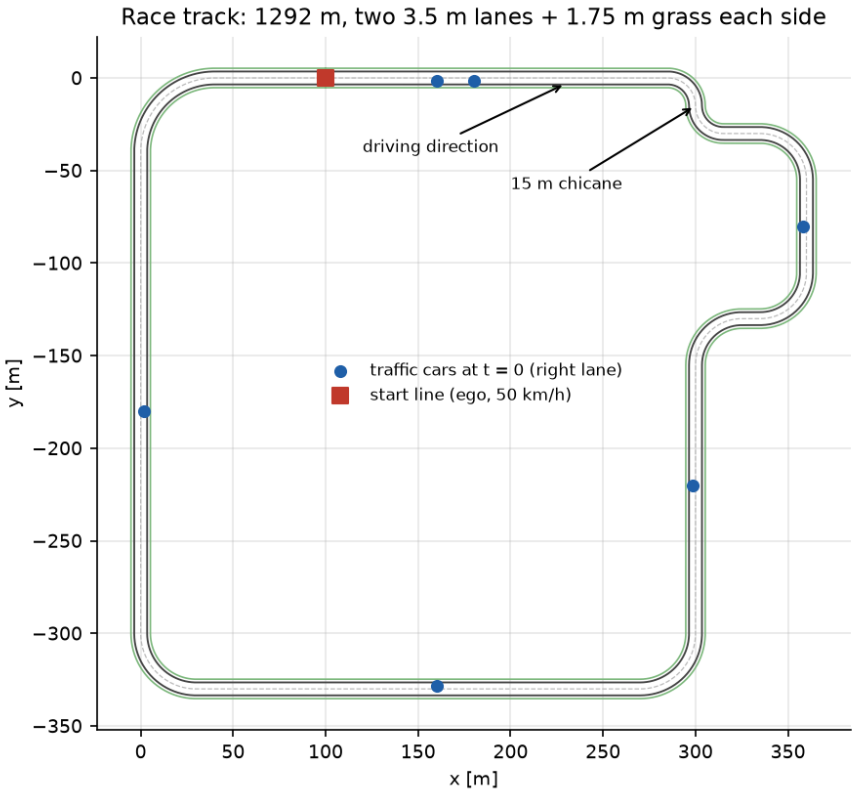


Figure 1. The circuit (config/racetrack.json): centreline = lane divider (dashed), asphalt edges (dark), grass edges (green), traffic cars at t = 0 (blue), start line (red). Driven clockwise; corner radii 15-40 m.

Item	Value
Track	1292 m, eight corners (40, 15, 15, 25, 25, 25, 30, 30 m radius), two 3.5 m lanes, 1.75 m grass each side (friction 0.9 asphalt / 0.3 grass), 1 m barrier beyond the grass in CARLA
Ego car	starts on the start line, right lane, 50 km/h; speed limit 80 km/h; one lap
Traffic	six cars on the right-lane centre at constant speed (one at 30 km/h, five at 50 km/h), no dynamics
Sensing (default)	traffic seen 60 m ahead / 20 m behind with noise (0.5 m along, 0.2 m across, 0.5 m/s), noisy ego state, road shape known 40 m ahead; --perfect for exact
Controllers	NMPC: 4 s horizon, 0.1 s steps, dynamic bicycle model with Fiala tires in track coordinates, soft constraints for traffic (ellipses), road edges, sideslip and speed. Pure pursuit: lookahead on the lane line, grip-limited speed profile (40 % of mu g), lane-change rule to pass
Control period	50 ms; CARLA runs synchronously at the same step

3. Architecture and repository layout

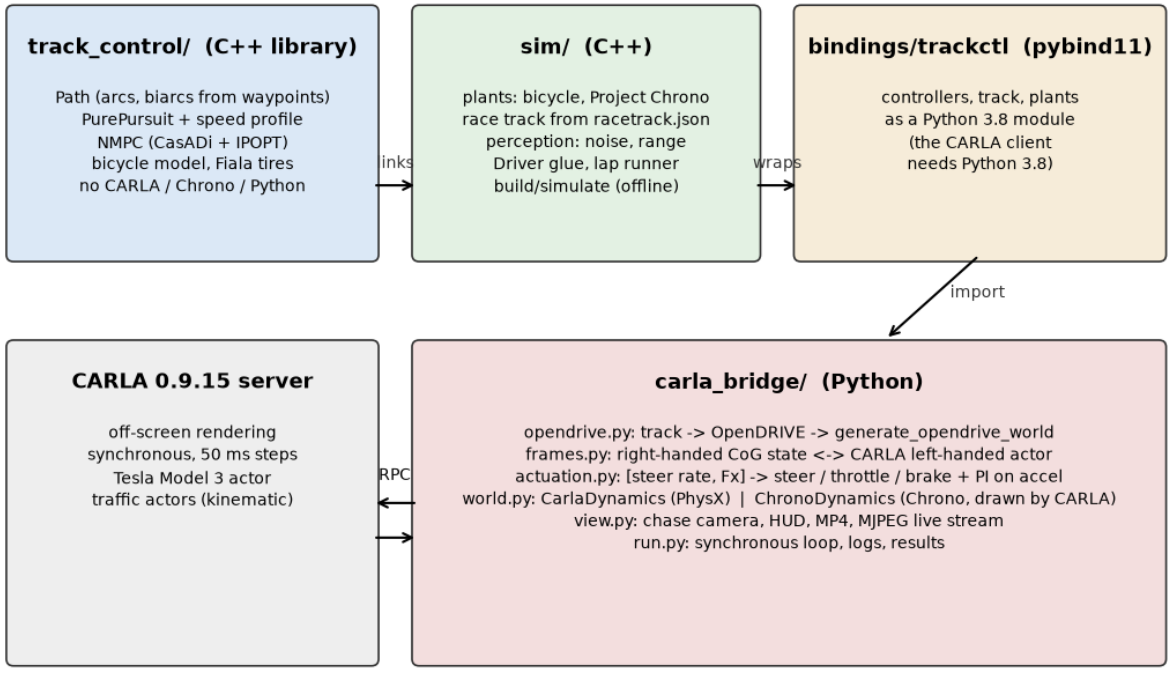


Figure 2. The C++ library at the core; the offline simulation and the Python module build on it; the CARLA bridge uses the module and talks to the CARLA server.

Path	What it holds
track_control/	The controller library (C++17, namespace tc): Path, Controller interface, PurePursuit, NMPC, speed profile, bicycle model and Fiala tire, VehicleParams; CMake package, tests, example, README
sim/plant/	Bicycle plant (the model plus steering lag and tire relaxation), Project Chrono plant (Sedan_Vehicle, PAC2002 tires), identify_chrono
sim/racetrack/	Circuit (read from JSON), perception (noise, ranges), Driver (controller + perception), lap runner and metrics
sim/simulate.cpp	build/simulate: offline laps on either plant
bindings/trackctl.cpp	pybind11 module for the bridge
carla_bridge/	opendrive.py, frames.py, actuation.py, world.py (dynamics backends), view.py (camera, HUD, video, stream), run.py (main loop, CLI)
config/	racetrack.json, vehicles/model3.json, vehicles/chrono_sedan.json, tuning.json (NMPC margins per dynamics)
scripts/	install_chrono.sh, install_carla.sh, start_carla.sh, env.sh, fit_chrono_tires.py
tests/	bridge tests and the stand-in carla module (tests/fake_carla)
setup.sh, run.sh	install everything; run (starts CARLA if needed)
.github/workflows/ci.yml	build + fast tests on every push (not yet run on GitHub)

4. The controller library: track_control

The library was refactored out of the earlier code so that it knows nothing about the race track, the perception model or any simulator. A controller is built from four things and called once per period:

Type	Role
tc::Path	Reference line as constant-curvature pieces (straights and arcs), parametrised by arc length; closed (lap) or open. Built from corner radii (from_corners), from waypoints (from_waypoints: a biarc between each pair of points, so the path passes through every point with continuous heading; meant for CARLA routes), or from pieces. Provides pose(s, e_y), project(x, y), curvature, offset-line lengths

Type	Role
tc::Road	Lane width, distance from the path to the drivable edge, friction, speed limit
tc::VehicleParams	Single-track parameters (mass, inertia, axle distances, cornering stiffnesses, friction per axle, brake and drive split and limits, drag, steering limits); defaults are CARLA's Tesla Model 3; from_file reads JSON
tc::Body	Body length, width, centre-of-gravity offset (road-edge and clearance constraints)
tc::Observation	Time, measured state [X, Y, psi, Ux, Uy, r, delta] at the centre of gravity in a right-handed frame, and obstacles {s, e_y, speed} in path coordinates
compute(obs)	Returns [steering rate, total longitudinal tire force], within the vehicle's limits

Design choices worth knowing:

- The model equations are templates evaluated on doubles (simulation) and on CasADi symbols (NMPC), so the plant and the controller share one implementation.
- The NMPC header hides CasADi behind a private implementation pointer: users of the library need CasADi only to link, not to compile against it.
- The output is a steering *rate* and a tire *force*, simulator-neutral; converting them to a simulator's pedals and steering is the integration's job (carla_bridge/actuation.py shows it for CARLA).
- CasADi from its pip wheel uses the old libstdc++ string ABI, so the library compiles with _GLIBCXX_USE_CXX11_ABI=0 by default (option TRACK_CONTROL_OLD_CXX_ABI).
- New in the model: a peak friction per axle (grip_f, grip_r, optional in JSON, default 1) - needed for the Chrono car (section 6). With the defaults the equations are numerically identical to before.
- Pure pursuit now clips its outputs to the steering-rate and force limits (it could ask for 68 kN from a standing start; the plants had been clipping it).

Using it from another project was checked by installing it and building a separate consumer with `find_package(track_control)`. To publish it with its history: `git subtree split --prefix=track_control -b track_control` and push that branch to a new repository.

5. CARLA integration and the two vehicle dynamics

5.1 The track in CARLA

The circuit is written as an OpenDRIVE road at start-up and loaded with `client.generate_opendrive_world`, so no map download is needed and the road CARLA draws is exactly the path the controllers follow: each straight is a <line> and each corner an <arc> (a test checks every 0.5 m of the reference line against the track). Lanes outward from the centreline: 3.5 m asphalt, then 1.75 m grass. CARLA's source was read to settle two details: it meshes every lane (non-driving types as raised sidewalks, so the grass lanes are typed 'driving'), and it puts a wall at the outer edge of the outermost lanes (1 m high here, a barrier).

5.2 Frames

The library works in a right-handed frame at the centre of gravity; CARLA is left-handed (y to the right, yaw clockwise, degrees) and reports an actor's origin, which for the Model 3 is 0.45 m behind the centre of mass (read from its physics control). frames.py converts position, heading, body velocities and yaw rate both ways; the lateral velocity is shifted by yaw rate times 0.45 m. OpenDRIVE is right-handed and CARLA flips y when it loads it, so track coordinates go into the OpenDRIVE file unchanged.

5.3 The two dynamics backends

	--dynamics carla	--dynamics chrono
Who moves the car	CARLA's PhysX vehicle (vehicle.tesla.model3)	Project Chrono Sedan_Vehicle (multibody chassis and suspension, PAC2002 tires, driveline), stepped in the bridge at 1 ms
CARLA's role	simulates and renders	renders only: ego physics off, pose written every step including Chrono's roll, pitch and heave
Controller model	config/vehicles/model3.json (read/derived from CARLA, partly assumed)	config/vehicles/chrono_sedan.json (identified from the Chrono model, tires fitted per axle)
Inputs	steering rate integrated to an angle, sent as a fraction of the wheels' max angle (CARLA's speed-dependent steering curve flattened); force to throttle or brake with a PI loop on acceleration that absorbs the engine map	steering rate and force directly (Chrono plant's own actuator model: 50 ms steering lag, axle torque, brakes split by brake_front)
Grass friction	not modelled (CARLA's surface is uniform)	0.3 on the grass, updated every 10 ms
NMPC margins (tuning.json)	mu_use 0.9, drive_use 0.25 - a cautious guess, untuned	drive_use 0.25 - measured (section 6)

CARLA runs in synchronous mode at the 50 ms control period with 10 ms physics sub-steps, so an NMPC solve that takes longer than 50 ms slows the simulation but does not change it. The bridge draws the NMPC's predicted path in the world, attaches a chase camera, overlays a HUD (controller, dynamics, time, lap progress, speed, lateral offset, sideslip, solve time, track map) and writes MP4 and/or an MJPEG stream.

6. Making the NMPC drive the Chrono car

On the Chrono plant the NMPC, planning with the first set of identified parameters, slid wide and left the track 13 s into the lap, in the first 15 m chicane; pure pursuit (which uses only 40 % of the grip) lapped fine. The investigation, in order:

What was tried (NMPC on Chrono, default sensing)	Result
First parameters (single mu 1.117 for both axles)	left the track at 13.0 s, sideslip 50 deg
Front-biased brake split 0.6 / 0.7 / 0.8 (plant and model)	left the track at 12.4 / 11.3 / 13.3 s
Friction margin mu_use 0.7 / 0.8, with brake caps	0.7: collision or off the track; 0.8: laps (68-70 s) but sideslip 23-48 deg, over the asphalt edge
Tighter sideslip limit 2-5 deg	erratic: some laps with 17-27 deg sideslip, others off the track
Yaw inertia x1.5, x2	left the track at 13 s
Lower speed limit 50 / 60 / 70 km/h	50: 93.6 s but 9 deg sideslip; 60: 31 deg; 70: off the track
Measurement: slow steering ramps on Chrono vs the bicycle model	Chrono reaches its limit by spinning (sideslip -11 to -12 deg) where the model understeers to 1.05 g with ~0 sideslip: the model's axle balance is wrong
Per-axle Fiala fit (front C 105 kN/rad, mu 1.151; rear C 152 kN/rad, mu 1.037)	laps on seed 0 (68.9 s) but 18 deg sideslip; collisions on seeds 1 and 2
Observation: in every remaining slide the NMPC commands full front-wheel drive	the friction circle says drive force takes front lateral grip (a yaw correction); Chrono's PAC2002 tires lose much less, so the correction does not happen
Fit + plan with half the drive force	laps on 3 seeds, 68.8-68.9 s, sideslip 7.6-8.1 deg, 0.04-0.08 m from the edge
Fit + plan with a quarter of the drive force (final)	**laps on 3 seeds, 70.2-70.6 s, sideslip <= 7.0 deg, 0.19-0.23 m from the edge; exact sensing 66.3 s**

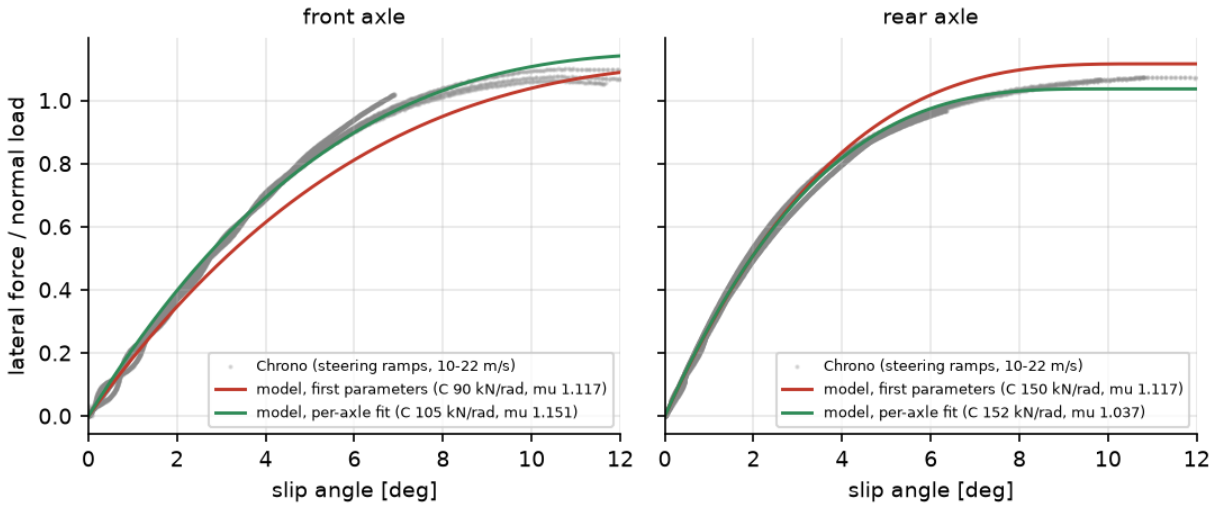


Figure 3. Axle lateral force against slip angle: Chrono (grey, from the force and moment balance during steering ramps at 10-22 m/s on friction 0.9) and the Fiala model before (red) and after (green) the per-axle fit. The first parameters made the front too weak and the rear too strong: the model expected understeer where the Chrono car is near-neutral and lets its rear go first. RMS error fell from 535 / 226 N to 174 / 134 N (front / rear). Reproduce with scripts/fit_chrono_tires.py.

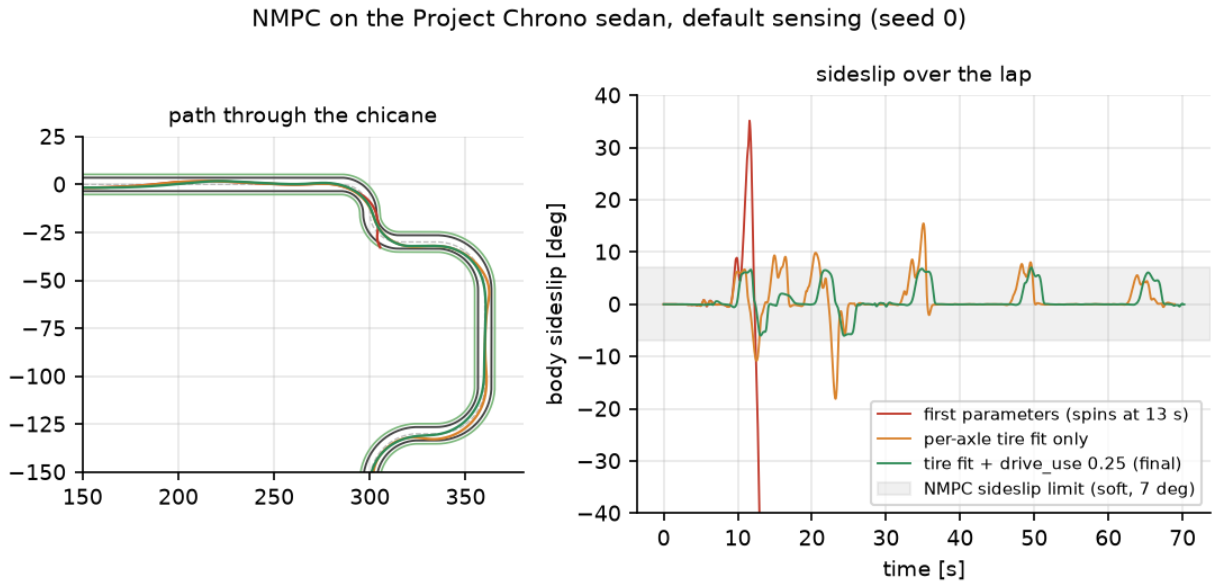


Figure 4. The NMPC on Chrono at each stage: first parameters (red, spins out at the chicane), tire fit only (orange, completes but slides past the 7 deg sideslip limit), tire fit with a quarter of the drive force (green, stays within the limit all lap).

Both fixes are about the model the NMPC plans with, not about tuning the controller; they are recorded in the parameter file (grip_f, grip_r, with their source) and in config/tuning.json (drive_use, with the measurements behind it). A better combined-slip tire model would let the NMPC use the full drive force; that is a possible next step.

7. Results (offline, the same controllers and plants without CARLA)

Plant	Controller	Sensing	Lap	Speed	Cars passed	Clearance	Asphalt margin	Peak sideslip
bicycle	NMPC	default	72.2 s	44-70 km/h	2/6	0.86 m	0.15 m	7.1 deg
bicycle	pure pursuit	default	98.5 s	24-61 km/h	1/6	1.34 m	0.11 m	6.8 deg
bicycle	NMPC	exact	65.2 s	47-80 km/h	3/6	0.72 m	0.19 m	7.0 deg
bicycle	pure pursuit	exact	90.5 s	24-80 km/h	1/6	1.24 m	0.12 m	5.8 deg
Chrono	NMPC	default	70.2 s	49-72 km/h	3/6	0.86 m	0.19 m	7.0 deg
Chrono	pure pursuit	default	98.2 s	24-61 km/h	1/6	1.29 m	0.19 m	5.1 deg
Chrono	NMPC	exact	66.3 s	50-80 km/h	3/6	0.68 m	0.11 m	7.1 deg
Chrono	pure pursuit	exact	90.4 s	24-80 km/h	1/6	1.00 m	0.21 m	4.7 deg

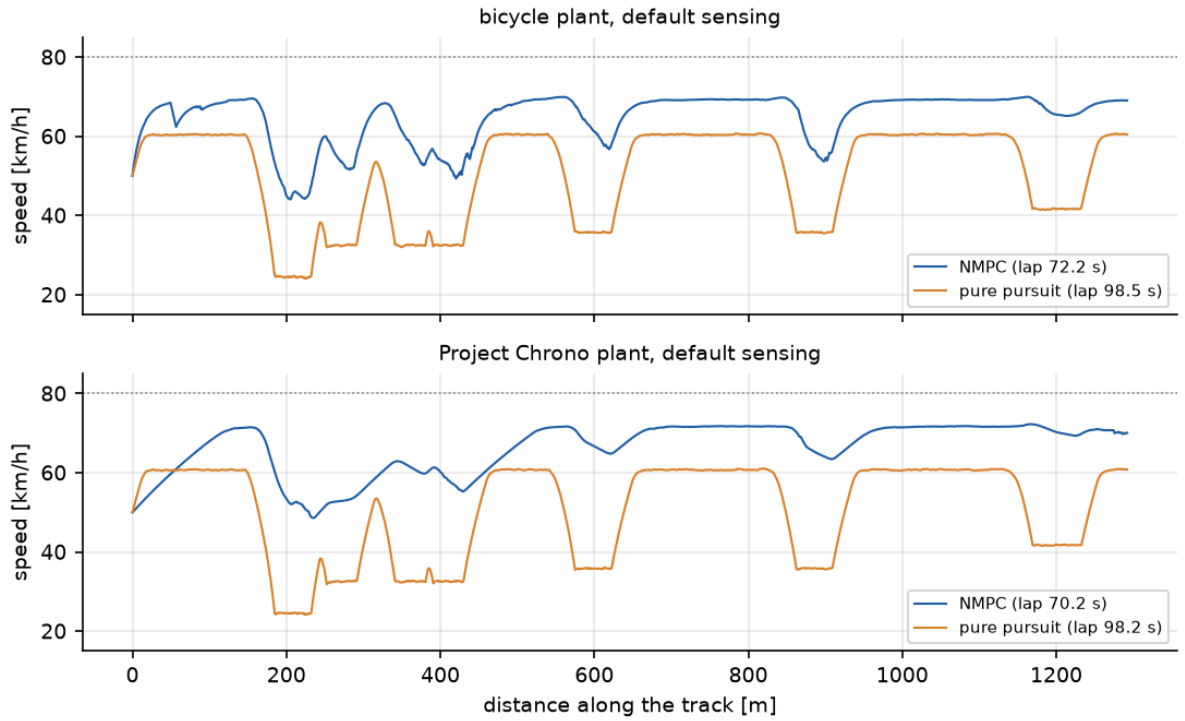


Figure 5. Speed along the lap. The NMPC finds the cornering speed from its tire model; pure pursuit follows a profile at 40 % of the grip. With the road known only 40 m ahead, both slow for what might come into view.

NMPC solves averaged 55-135 ms in these runs (several running at once on one machine), above the 50 ms period; a few solves per lap on Chrono end without converging (11 in the default run) and their result is applied anyway. Chrono is built with -march=native, so its numbers vary slightly between machines: the exact-sensing NMPC lap was 66.3 s here (GCC 13) and 66.4 s in the clean container (GCC 11).

8. How it was verified

Check	Outcome
Refactor regression: bicycle-plant laps against the original project's build/simulate (both controllers, default and exact sensing)	identical, every sample of all 13 logged columns
Library tests (ctest): paths (corners, biarc waypoints, open paths, projection), model (steady yaw rate vs understeer formula, force limits, JSON), controllers in closed loop (following, passing)	3 of 3 pass
Bridge tests against the stand-in carla module (CARLA conventions: left-handed, degrees, actor origin, pedals, an engine giving 85 % of the asked force; bicycle physics)	8 of 8 pass: OpenDRIVE geometry, frames round trip, actuation signs, video file, live stream, full laps with pure pursuit and NMPC on 'CARLA' dynamics and pure pursuit on Chrono dynamics
Installed library used from a separate CMake project via find_package	builds and runs
setup.sh --no-carla from scratch in a clean ubuntu:22.04 container (Chrono build included), then an offline NMPC lap on Chrono and two bridge laps	passes; repeated with the final files (--no-chrono)
Build without Chrono	builds; the Chrono plant reports how to install Chrono

9. Not yet verified, and what to expect on the first Brev run

Nothing here has driven a real CARLA server. Items that may need attention:

Item	Why it is uncertain	Where to adjust
CARLA server start on a headless VM	Vulkan off-screen rendering on cloud GPUs is a known trouble spot (CARLA issue 9492, Bench2Drive issue 151)	scripts/start_carla.sh log; --docker as an alternative
Actuation loop	gains chosen, not tuned against CARLA's engine and brakes	carla_bridge/actuation.py (kp, ki)
Model 3 parameters	yaw inertia, CoG height, friction, drag area, 4WD split are assumptions (docs/vehicle_parameters.md)	config/vehicles/model3.json
NMPC margins for CARLA	mu_use 0.9, drive_use 0.25 copied cautiously from the Chrono experience	config/tuning.json, no rebuild needed
Velocity reference point	assumed CARLA reports the actor origin's velocity (0.45 m behind the CoG); if not, lateral velocity is off by yaw rate x 0.45 m	carla_bridge/frames.py
Chrono roll sign when drawn	visual only	carla_bridge/world.py transform()
GitHub CI workflow	written, not yet run on GitHub	.github/workflows/ci.yml

10. Command reference

```
./setup.sh [--no-carla] [--no-chrono] [--no-system] [--jobs N]
./run.sh [--controller nmpc|pure_pursuit|both] [--dynamics carla|chrono] [--perfect]
         [--road-range M] [--seed N] [--laps N] [--t-max S] [--stream PORT] [--no-video]
         [--no-plan] [--docker] [--quality Low|Epic]
./run.sh --stop-carla
./run.sh --offline [build/simulate options]       # no CARLA

build/simulate [--controller both|nmpc|pure_pursuit] [--plant bicycle|chrono] [--perfect]
               [--range M] [--noise K] [--ego-noise K] [--road-range M] [--seed N]
               [--speed-limit KMH] [--mu-use K] [--brake-use K] [--drive-use K] [--sideslip DEG]

ctest --test-dir build                            # library tests
.venv/bin/python -m pytest [-m "not slow"]        # bridge tests (stand-in CARLA)
.venv/bin/python scripts/fit_chrono_tires.py      # refit the Chrono tires
```

Suggested next steps

- First Brev run: confirm the server starts, then run pure pursuit on CARLA dynamics (the most forgiving combination) before the NMPC; tune the actuation gains from the logged commanded vs measured acceleration.
- Identify the Model 3 in CARLA the way the Chrono car was (steering ramps, per-axle tire fit) and revisit the CARLA margins in tuning.json.

- A combined-slip tire model in the NMPC, so it can use the full drive force on Chrono.
- Bring the NMPC solve time under the 50 ms period (e.g. an SQP/RTI scheme); split track_control into its own repository and tag a first release.